

ECS 160 – Memory and Attacks

Instructor: Tapti Palit

Teaching Assistant: **Xingming Xu**



Recap: Memory

- We store everything about our programs in memory
 - Program data, variables, etc.
 - Code instructions
 - Control information; the flow of the program
- Mistakes in memory management can
 - Lead to crashing
 - Leak information altogether
 - Allow code execution (control flow hijack)



Recap: C Pointers

- When a variable is created, a memory address is assigned to it
- A **pointer** stores that memory address as its value
- Java has references to manage objects in memory
 - As opposed to unmanaged pointers



Recap: Buffer Overflows

- **Buffer** – memory set aside to hold some data
- Buffer size is usually fixed in C
 - `char* p* = malloc(128);`
 - Which C itself doesn't have any way to check
- `p[1000]` would check the part of memory the compiler believes points at the 1000th element of an array `p`
 - Regardless of what that actually is
- **Buffer overflow** – program writes data outside expected memory bounds, causing erratic behavior

Causes

- Coding errors and mistakes in app development
- Especially problematic in C, C++
- Many **interpreted languages** are memory safer
 - Java, Rust, Go
- Still happens in C/C++ powered low-level systems



Java's Fixes

- Eliminates some of the most common memory bugs
- Automatic bound checking – adhere to object boundaries
- Garbage collection
- No exposed pointers
- Expected behavior vs undefined behavior



Memory Layout (in C)

- The OS will give each process some allocated memory
- From high to low memory addresses...
- Allocate function-local variables on the **stack**
- Malloc allocates via the **heap**
- Global, static variables assigned to a “data” section
- Code, instructions are reserved some space at the very “end”



Memory Performance Tradeoff

- Bound checks are expensive to run
- Additional checks will reduce performance
- Thus, systems requiring quick performance will use C/C++ anyways

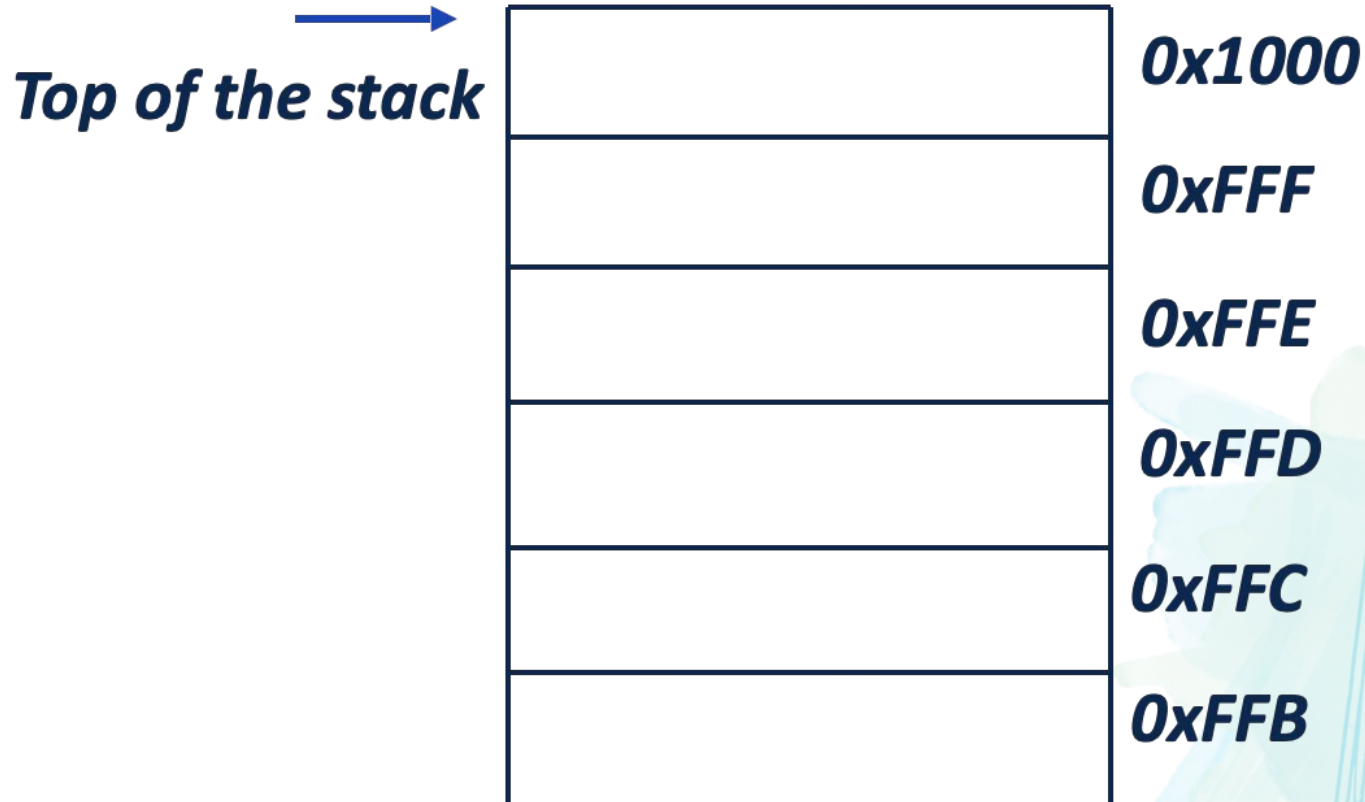


C Stack Layout

- Stack contains local variables, function parameters, return addresses for function calls
- The stack grows downward in memory
 - There is some historical rationale behind this...



Stack Growth



Example (Lecture 9, slide 31)

- Function local variables are stored on stack

```
1. int my_func(int a, int b) {  
2.   int result = 0;  
3.   if (a > b) { result = a - b; }  
4.   else { result = b - a; }  
5.   return result;  
6. }
```

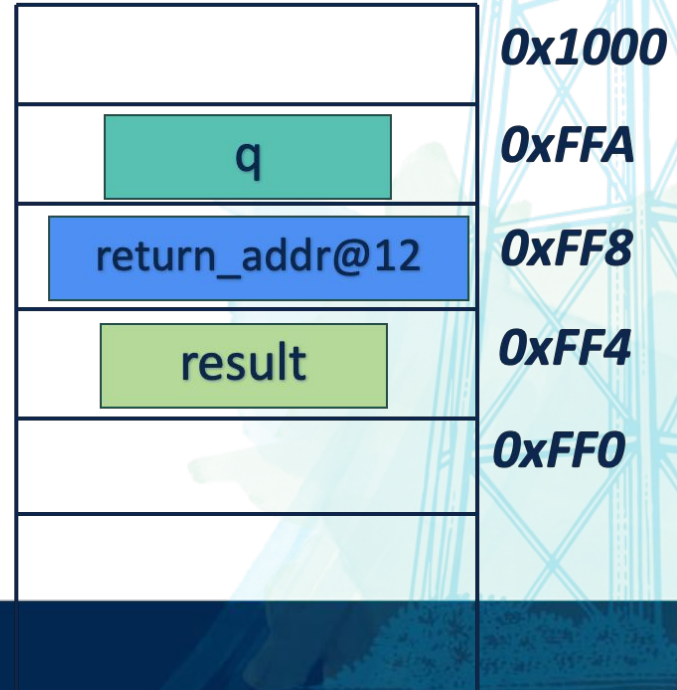
```
8. int main(void) {  
9.   int p = 10;  
10.  int q = 30;  
11.  int sub = my_func(p, q);  
12. }  
13.
```

Top of the stack

*Stack frame for
main*

*Stack frame for
my_func*

On x86, stack grows downwards



Return Address

- Return addresses for a given function live “above” in memory than functional local variables
- The memory growing downwards is “redirected upwards”



Hijack Control Flow

- If an attacker can overwrite static variables, they can end up with access to the stack variables via overflow
- Can maliciously change program execution
- Since the program is technically just jumping from address to address
 - If a malicious actor can get access to the memory used for functions
 - Worst case: execute arbitrary code starting from there

Preventing This

- Avoid using unsafe, “legacy-support” functions
- strcpy
- strcat
- scanf



In Java

- Fortunately, because these “basic” memory issues have been addressed, we don’t really have to deal with attacks like *these*
 - Though other vulnerabilities are still present



Temporal Weaknesses in C

- Compilers don't detect updates to free memory on the **heap**
 - **Use after free** – accessing a memory location after it has been freed
- A running program needs to be aware of where it is actually free
- Dangling pointers
 - Pointers pointing to a memory location that has been deallocated

Attacking via UAF

- Since the C compiler won't recognize if memory has been freed twice...
 - If you can gain access to that dangling pointer, you can reference that part of the memory
- Generally, this is undefined behavior
- Worst case: remote execution of code
- To prevent: In C++, set pointers to nullptr after usage
 - In C, be careful